

1. Code

I: Kernel EigenFaces

Data Loading: The function reads all images from training and testing folders. Converts images to grayscale, resizes each image to 50×50 then flattens each image into a 2500-dimensional vector. Extracts labels from filename (subject ID).

```
def CreateMatrix(path):
    path = Path(path)
    X, Y = [], []

    for file in sorted(path.iterdir()):
        if file.is_file():
            img = cv2.imread(str(file), cv2.IMREAD_GRAYSCALE)
            img = cv2.resize(img, (50, 50))
            X.append(img.flatten())

            label = int(file.stem.split(".")[0].replace("subject", ""))
            Y.append(label)

    return np.array(X, dtype=np.float64), np.array(Y)
```

Applying PCA and LDA (14 components for 15 classes):

PCA: each image is mean-centered and PCA is performed using the covariance matrix .

Eigenvectors are computed from $Cv = \lambda v$ and then mapped back to image space as eigenfaces.

The final projection is $Z = (X - \mu)W$, which is used for classification and reconstruction.

```
class PCA:
    def __init__(self, n_components=100):
        self.n_components = n_components
    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        Xc = X - self.mean
        cov = np.dot(Xc, Xc.T)
        eigvals, eigvecs = np.linalg.eigh(cov)
        idx = np.argsort(eigvals)[::-1]
        eigvecs = eigvecs[:, idx]
        eigenfaces = np.dot(Xc.T, eigvecs)
        eigenfaces = eigenfaces / (np.linalg.norm(eigenfaces, axis=0) + 1e-8)
        self.components = eigenfaces[:, :self.n_components]
    def transform(self, X):
        return np.dot(X - self.mean, self.components)
    def reconstruct(self, X):
        return np.dot(self.transform(X), self.components.T) + self.mean
```

LDA: we compute within-class scatter $S_w = \sum (X_c - \mu_c)^T (X_c - \mu_c)$ and between-class scatter $S_b = \sum n_c (\mu_c - \mu) (\mu_c - \mu)^T$. It then solves S_w^{-1} to find optimal discriminant directions. The final projection $Z = XW$ is used for classification with better class separation.

$$S_w^{-1} S_b v = \lambda v$$

```
class LDA:
    def __init__(self, n_components=10):
        self.n_components = n_components
    def fit(self, X, y):
        self.mean = np.mean(X, axis=0)
        classes = np.unique(y)
        Sw = np.zeros((X.shape[1], X.shape[1]))
        Sb = np.zeros((X.shape[1], X.shape[1]))
        for c in classes:
            Xc = X[y == c]
            mean_c = np.mean(Xc, axis=0)
            Sw += (Xc - mean_c).T @ (Xc - mean_c)
            n = Xc.shape[0]
            diff = (mean_c - self.mean).reshape(-1, 1)
            Sb += n * diff @ diff.T
        eigvals, eigvecs = np.linalg.eig(np.linalg.pinv(Sw + 1e-8 * np.eye(Sw.shape[0])) @ Sb)
        idx = np.argsort(eigvals)[::-1]
        self.components = np.real(eigvecs[:, idx[:self.n_components]])
    def transform(self, X):
        return np.dot(X - self.mean, self.components)
```

Reconstructing the Eigenfaces/Fisherfaces:

Function `def reconstruct(self, X):` for PCA code reconstructs face images by projecting them into the PCA (eigenface) space $Z = (X - \mu)W$ and then mapping them back to the original image space using the inverse transformation, allowing comparison between original and compressed representations.

```
    return np.dot(X - self.mean, self.components)
def reconstruct(self, X):
    return np.dot(self.transform(X), self.components.T) + self.mean
```

LDA CALSS

Fisherfaces are the projection directions learned by LDA that best separate different face classes. They are the basis vectors that maximize class separation. They Solve:

$$S_w^{-1} S_b v = \lambda v$$

Inside the LDA class we have:

```
eigvals, eigvecs = np.linalg.eig(np.linalg.pinv(Sw + 1e-8 * np.eye(Sw.shape[0])) @ Sb)
idx = np.argsort(eigvals)[::-1]
self.components = np.real(eigvecs[:, idx[:self.n_components]])
```

Part 2:

Classification & kernel methods:

KNN classifies each test sample by finding the k nearest training samples using Euclidean distance and assigning the majority class label among them.

$$d(x, x_i) = ||x - x_i||$$

$$\hat{y} = \text{mode}(y_{k\text{-nearest}})$$

```
class KNN:
    def __init__(self, k=3):
        self.k = k
    def fit(self, X, y):
        self.X = X
        self.y = y
    def predict(self, x):
        preds = []
        for x in x:
            d = np.linalg.norm(self.X - x, axis=1)
            idx = np.argsort(d)[:self.k]
            preds.append(Counter(self.y[idx]).most_common(1)[0][0])
        return np.array(preds)
```

Kernel PCA computes a kernel similarity matrix using either linear or RBF kernels and performs eigen decomposition to find principal directions in kernel space. The data is then projected using the learned eigenvectors. $K_{i,j} = k(x_i, x_j)$ and $Z = K\alpha$

```
class KernelPCA:
    def __init__(self, kernel="rbf", n_components=25):
        self.kernel = kernel
        self.n_components = n_components
    def _kernel(self, X, Y):
        if self.kernel == "rbf":
            return rbf_kernel(X, Y)
        elif self.kernel == "linear":
            return linear_kernel(X, Y)
        else:
            raise ValueError("Unknown kernel")
    def fit(self, X):
        self.X = X
        K = self._kernel(X, X)
        eigvals, eigvecs = np.linalg.eigh(K)
        idx = np.argsort(eigvals)[::-1]
        eigvecs = eigvecs[:, idx]
        self.alphas = eigvecs[:, :self.n_components]
        for i in range(self.alphas.shape[1]):
            self.alphas[:, i] /= np.linalg.norm(self.alphas[:, i]) + 1e-8
        self.train_kernel = K
```

Kernel LDA builds a kernel matrix and approximates within-class and between-class scatter matrices in kernel space. It then solves an eigenvalue problem to find directions that maximize class separability.

```
class KernelLDA:
    def __init__(self, kernel="rbf", n_components=10):
        self.kernel = kernel
        self.n_components = n_components
    def _kernel(self, X, Y):
        if self.kernel == "rbf":
            return rbf_kernel(X, Y)
        elif self.kernel == "linear":
            return linear_kernel(X, Y)
        else:
            raise ValueError("Unknown kernel")
    def fit(self, X, y):
        self.X = X
        K = self._kernel(X, X)
        Z = np.ones((K.shape[0], K.shape[0])) / 9
        Sw = K @ K
        Sb = K @ Z @ K
        M = np.linalg.pinv(Sw + 1e-8 * np.eye(Sw.shape[0])) @ Sb
        eigvals, eigvecs = np.linalg.eigh(M)
        idx = np.argsort(eigvals)[::-1]
        self.alphas = np.real(eigvecs[:, idx[:self.n_components]])
        for i in range(self.alphas.shape[1]):
            self.alphas[:, i] /= np.linalg.norm(self.alphas[:, i]) + 1e-8
        self.train_kernel = K
```

PCA and LDA are used to reduce the dimensionality of face images and extract discriminative features. These features are then classified using KNN, where each test sample is assigned the label based on majority voting among its nearest neighbors. Performance is measured using accuracy on the test set.

```
knn = KNN(k=5)
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)
knn.fit(X_train_pca, y_train)
pca_acc = np.mean(knn.predict(X_test_pca) == y_test)

X_train_lda = lda.transform(X_train)
X_test_lda = lda.transform(X_test)
knn.fit(X_train_lda, y_train)
lda_acc = np.mean(knn.predict(X_test_lda) == y_test)
```

Part 3:

Kernel PCA and Kernel LDA are used to map face data into a higher-dimensional feature space

using linear and RBF kernels. KNN is applied later on these transformed kernel features for classification. The results are compared between PCA vs Kernel PCA, LDA vs Kernel LDA, and different kernels (linear vs RBF) to evaluate performance differences.

```
# kernel methods
for k in ["rbf", "linear"]:
    kpca = KernelPCA(kernel=k, n_components=50)
    kpca.fit(X_train)
    X_train_kpca = kpca.transform(X_train)
    X_test_kpca = kpca.transform(X_test)
    knn.fit(X_train_kpca, y_train)
    kpca_acc.append(np.mean(knn.predict(X_test_kpca) == y_test))

    kllda = KernelLDA(kernel=k, n_components=10)
    kllda.fit(X_train, y_train)
    X_train_kllda = kllda.transform(X_train)
    X_test_kllda = kllda.transform(X_test)
    knn.fit(X_train_kllda, y_train)
    kllda_acc.append(np.mean(knn.predict(X_test_kllda) == y_test))

print("\n===== FINAL RESULTS =====")
print(f"PCA Accuracy: {pca_acc:.3f}")
print(f"LDA Accuracy: {lda_acc:.3f}")
print(f"Kernel PCA (RBF): {kpca_acc[0]:.3f}")
print(f"Kernel PCA (Linear): {kpca_acc[1]:.3f}")
print(f"Kernel LDA (RBF): {kllda_acc[0]:.3f}")
print(f"Kernel LDA (Linear): {kllda_acc[1]:.3f}")
```

II: t-SNE

Symmetric SNE: Symmetric SNE, similarity in low-dimensional space is modeled using a Gaussian distribution:

$$Q_{ij} = \frac{\exp(-||y_i - y_j||^2)}{\sum_{k \neq l} \exp(-||y_k - y_l||^2)}$$

t-SNE : In t-SNE, similarity is modeled using a Student-t distribution (1 degree of freedom):

$$Q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

In SNE, Gaussian similarity decays too quickly in low dimensions. This causes many points to collapse into the center. Also distant points cannot be represented properly.

$$\exp(-||y_i - y_j||^2)$$

$$\frac{1}{1 + ||y_i - y_j||^2}$$

t-SNE replaces Gaussian with Student-t distribution:

This has heavy tails, keeps distant points far apart, reduces overcrowding in the center, and improves cluster separation.

Below code is changed to switch to symmetric SNE

```
else:
    for i in range(n):
        dY[i, :] = 4 * np.sum(np.tile(PQ[:, i], (no_dims, 1)).T * (Y[i, :] - Y), 0)
```

Visualization of Data Points:

Inside the `t_sne()` function below code is used to take a snapshot of `Y` every 10 iterations.

```
iY = momentum * iY - eta * (gains * dY)
Y += iY
Y -= np.mean(Y, axis=0)
if iter % 10 == 0:
    history.append(Y.copy())

if (iter + 1) % 10 == 0:
    C = np.sum(P * np.log(P / O))
```

This is used later to visualize the result. Below code is used to generate gif in [visualize.py](#) file using the frames generated each iteration.

```
def make_gif(history, labels, filename, folder):
    frames = []
    os.makedirs(folder, exist_ok=True)
    for i, Y in enumerate(history):
        fig, ax = plt.subplots(figsize=(6, 5))
        ax.scatter(Y[:, 0], Y[:, 1], c=labels, cmap="tab10", s=5)
        ax.set_title(f"Iteration {i * 10}")
        ax.set_xlim(-50, 50)
        ax.set_ylim(-50, 50)
        frame_path = os.path.join(folder, f"frame_{i}.png")
        plt.savefig(frame_path, dpi=150)
        plt.close(fig)
        frames.append(Image.open(frame_path))
    frames[0].save(filename, save_all=True, append_images=frames[1:], duration=300, loop=0)
    print("Saved:", filename)
```

Part 3:

Pairwise Similarities:

Pairwise similarity in original space:

```
P = x2p(X, 1e-5, perplexity)
P = P + P.T
P = P / np.sum(P)
P = P * 4.0
P = np.maximum(P, 1e-12)
```

Pairwise similarity in embedded space:

```
num = np.exp(-dist) # Gaussian (SNE)
num[range(n), range(n)] = 0.
Q = (num + num.T) / (2. * np.sum(num))
Q = np.maximum(Q, 1e-12)
Q_final = Q
PQ = P - Q
if method == "tsne":
```

Part 4:

Trying different perplexity values with list [2,20,50,65]

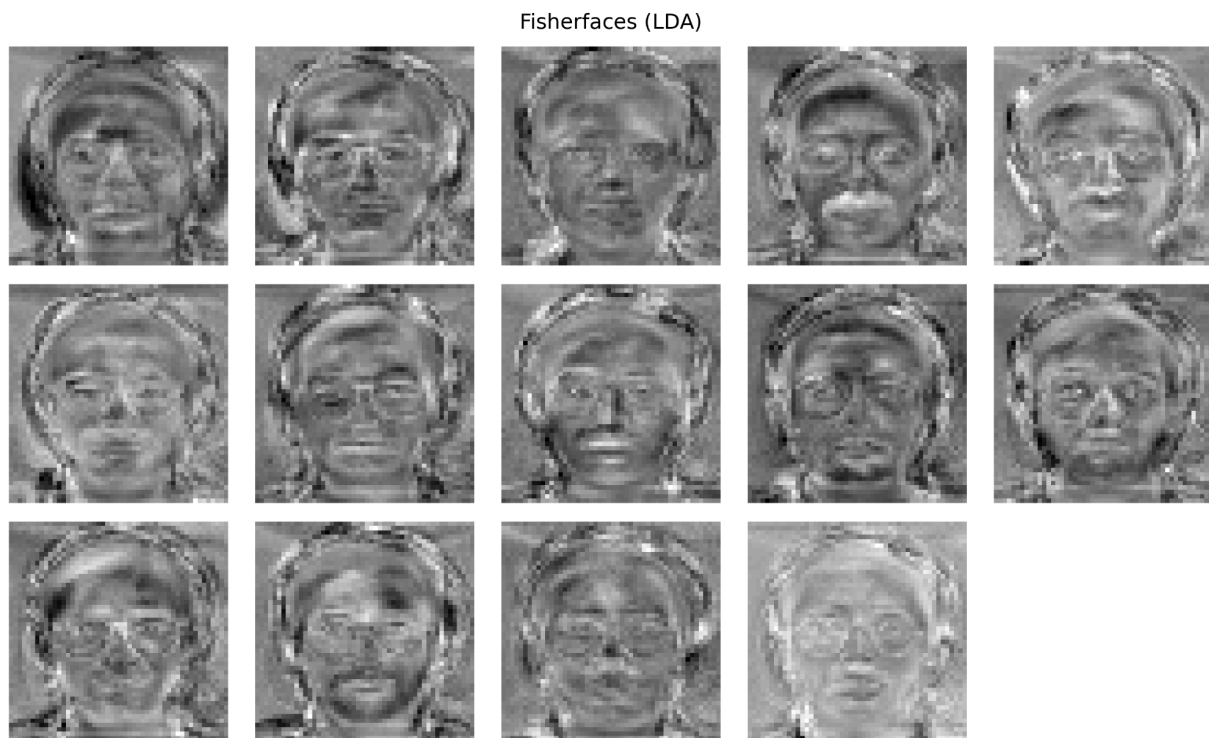
```
perplexity = [2.0,20.0,50.0,65.0]
for prep in perplexity:
    Y_tsne, P_tsne, Q_tsne, history_tsne = tsne(X, no_dims=2, initial_dims=50, perplexity=prep, method="tsne")
    Y_sne, P_sne, Q_sne, history_sne = tsne(X, no_dims=2, initial_dims=50, perplexity=prep, method="sne")
    np.save("./numpy_files/history_tsne.npy", np.array(history_tsne, dtype=object), allow_pickle=True)
```

2. Results and Experimental:

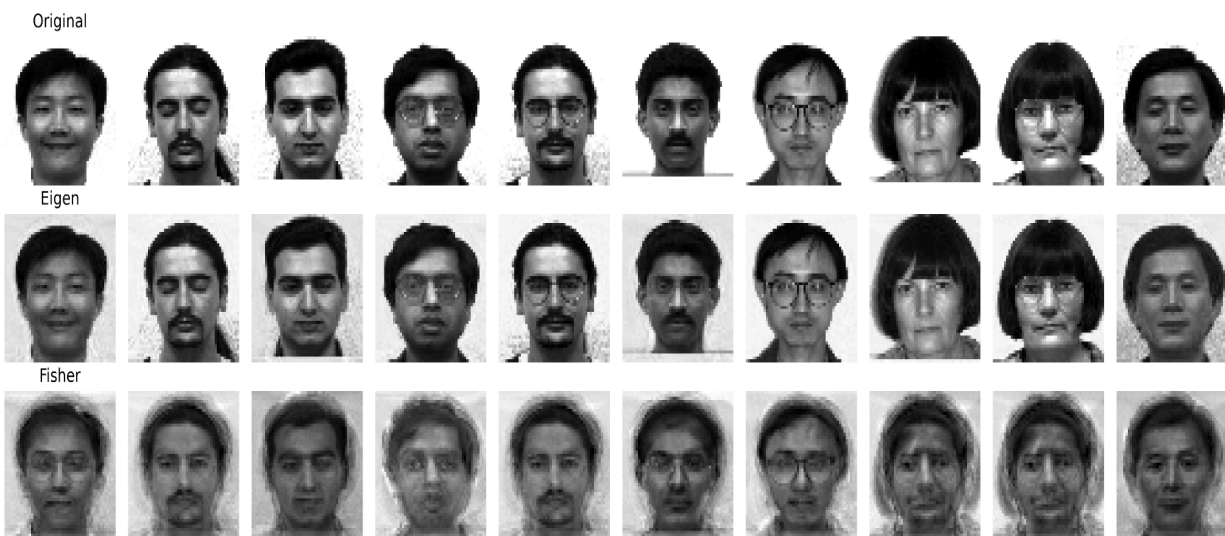
I: Kernel EigenFaces : Eigenfaces created:



Fisherfaces reconstructed:



Reconstructing the Eigenfaces/Fisherfaces:



Eigenfaces look clearer because PCA preserves general image structure, while Fisherfaces look blurrier because LDA focuses on class separation and discards more visual detail.

Part 3:

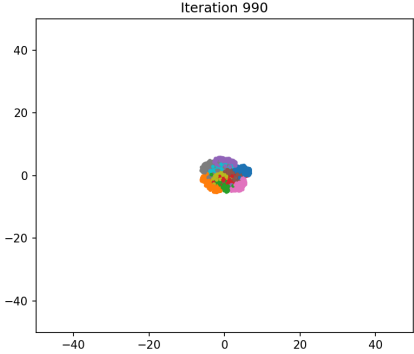
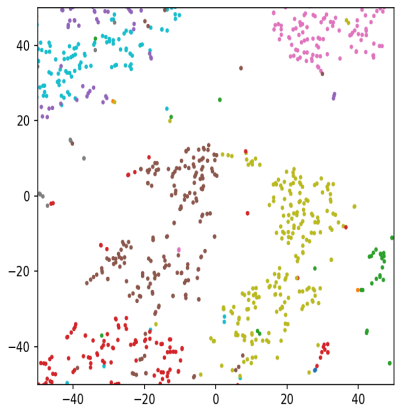
Performance Analysis:

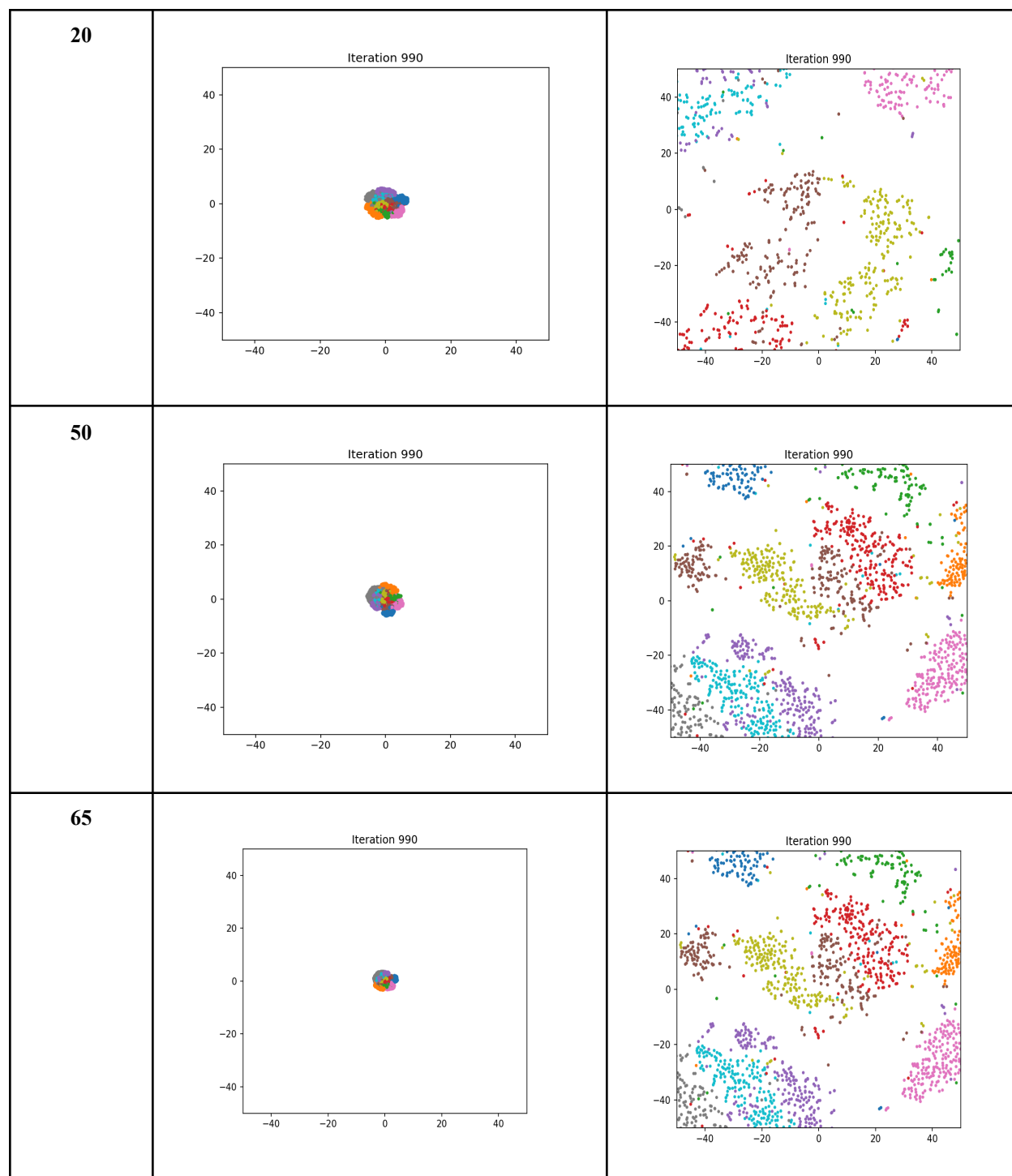
Model	Accuracy	Error
PCA	90%	10%
LDA	100%	0%
Kernel PCA (RBF)	86.7%	13.3%
Kernel PCA (Linear)	90%	10%
Kernel LDA (RBF):	83.3%	16.7%
Kernel LDA (Linear):	96.7%	3.3%

LDA works best because it knows the labels, so it learns how to separate the different people directly. PCA and Kernel PCA are a bit weaker because they only look at variation in the images, not which face belongs to which person. Kernel LDA is more sensitive, so depending on the data and settings it can do worse or better; here the linear version fits my data better than the RBF one.

II: t-SNE

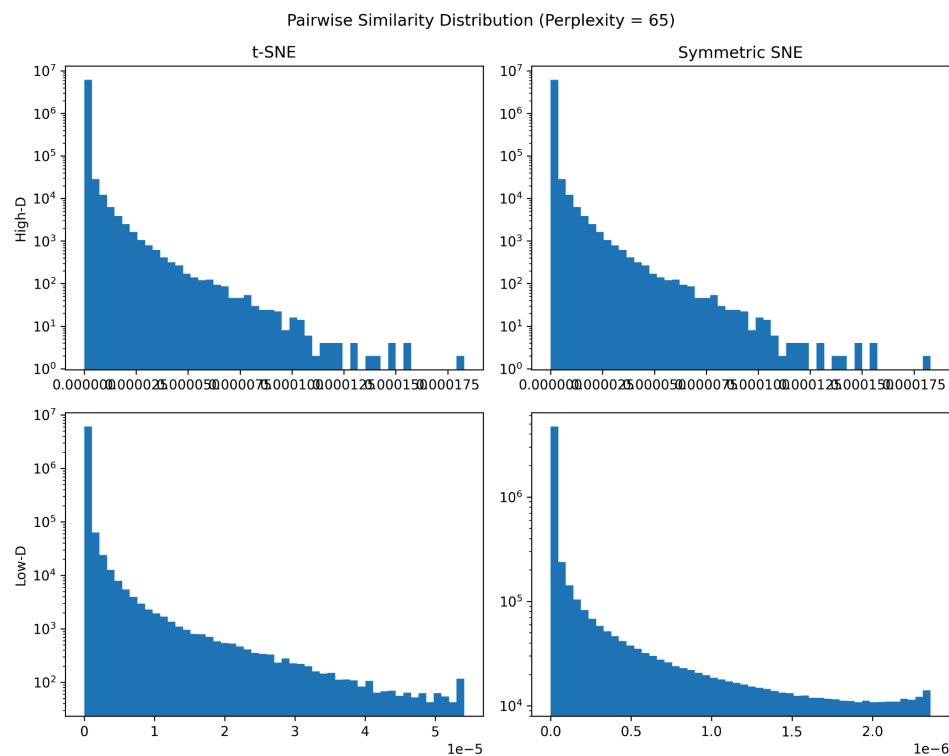
Visualization of Data Points in 2D space:

Perplexity	SNE	t-SNE
15	 Iteration 990 The SNE plot shows a single, dense cluster of data points centered around the origin (0,0). The axes range from -40 to 40. The points are colored by class, but they are all mixed together in a single blob.	 Iteration 990 The t-SNE plot shows the same data points as the SNE plot, but they are clearly separated into distinct clusters based on their color. There are approximately 10-12 distinct clusters visible, each representing a different class. The axes range from -40 to 40.



SNE collapses into a dense blob because the Gaussian similarities in low-dimensional space suffer from the crowding problem, causing many points to be mapped near the center. This behavior persists across all tested perplexities, indicating that changing perplexity alone is insufficient to separate the clusters.

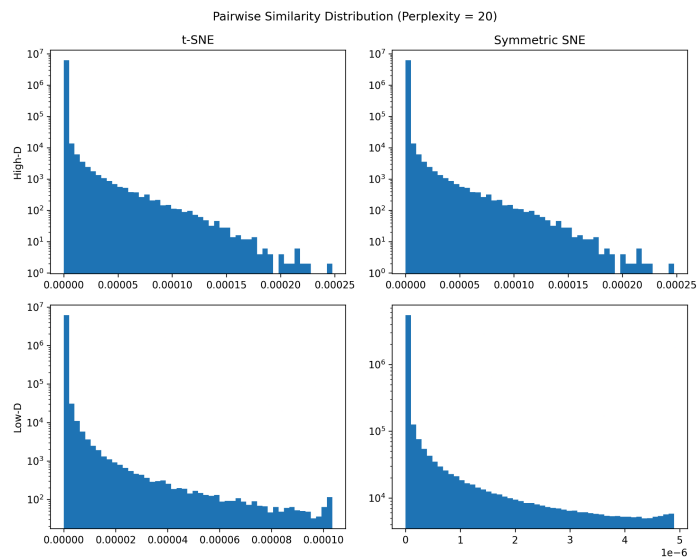
Part 4: Pairwise similarities in high D and low D , Perplexity : 65:



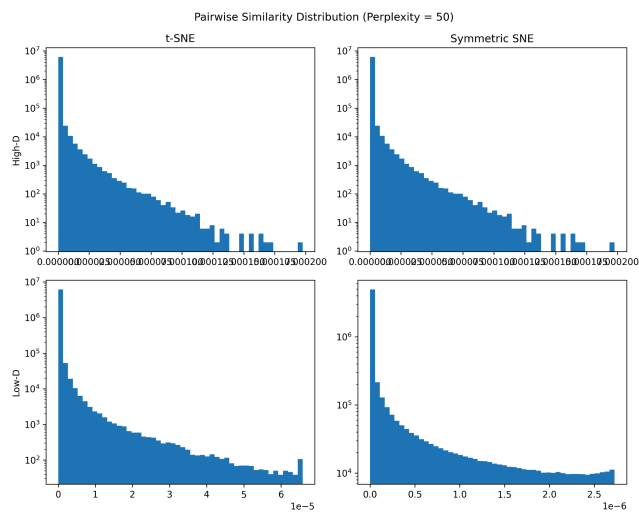
The crowding problem is observed because the low-dimensional similarity distribution is highly concentrated near zero compared to the high-dimensional distribution. This suggests that many distances are compressed in the embedding space, causing points to crowd together and reducing cluster separability.

Perplexity	Pairwise similarities
15	Pairwise Similarity Distribution (Perplexity = 15) The figure displays four histograms arranged in a 2x2 grid, comparing pairwise similarity distributions for High-D and Low-D embeddings using t-SNE and Symmetric SNE at a perplexity of 15. The y-axis for all plots is on a logarithmic scale from 10^0 to 10^7. The top row, labeled 'High-D', shows similarity distributions for t-SNE (left) and Symmetric SNE (right), with x-axes ranging from 0.00000 to 0.00025. The bottom row, labeled 'Low-D', shows similar distributions for the same methods, with x-axes ranging from 0.00000 to 0.00012. In all cases, the distributions are heavily right-skewed, indicating that most pairwise similarities are very low. The Low-D distributions are notably more concentrated near zero than the High-D distributions, which is a sign of the crowding problem.

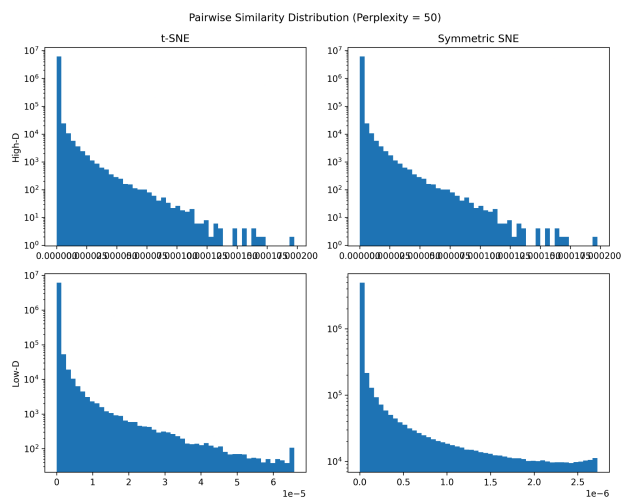
20



50



65



3. Conclusions:

1. In this homework, PCA, LDA, Kernel PCA, and Kernel LDA were implemented for face recognition. The results show that dimensionality reduction can significantly improve classification performance while reducing the complexity of the data.
2. Among all face recognition methods, LDA achieved the best result with 100% accuracy. This is because LDA uses class label information and focuses on separating different subjects, making it more effective than PCA for classification tasks.
3. For data visualization, t-SNE produced much clearer clusters than Symmetric SNE. The Student-t distribution used in t-SNE helps reduce the crowding problem and allows similar data points to stay together while keeping different clusters apart.
4. Different perplexity values were tested, and t-SNE consistently generated meaningful cluster structures. In contrast, Symmetric SNE often collapses into a dense blob, showing that t-SNE is a more robust and effective method for visualizing high-dimensional data in low-dimensional space.